Homework 3: Imbalanced Land-Cover Classification with Logistic Regression

Scientific Context

In this homework, you will work with the **Statlog Landsat Satellite** dataset. Each observation represents a 3 by 3 neighborhood of Landsat multispectral pixels, and the original class label describes the land-cover class of the center pixel.

We will convert the original multiclass problem into an **imbalanced binary-classification** problem:

> Can we identify the minority land-cover class, **damp grey soil**, from satellite spectral features?

This is a useful geoscience and environmental-data problem because remote-sensing models are often used to map rare or spatially limited surface conditions. In those settings, ordinary accuracy can be misleading: a model may look accurate while failing to detect the class we care about.

Learning Objectives

By completing this assignment, you will be able to:

1. Load and inspect a public earth-observation dataset in R.
2. Convert a multiclass land-cover problem into an imbalanced binary-classification problem.
3. Quantify class imbalance and explain why accuracy can be misleading.
4. Create stratified train, validation, and test sets.
5. Build leakage-safe preprocessing and logistic-regression workflows with `tidymodels`.
6. Compare a naive baseline, ordinary logistic regression, and upsampled logistic regression.
7. Evaluate imbalanced classifiers using recall, precision, specificity, balanced accuracy, F-scores, ROC-AUC, and PR-AUC.
8. Evaluate whether changing the probability threshold helps, using validation data only.

Submission

Submit this completed R Markdown document knitted to PDF. Replace every **TODO** with your own code or written answer. Keep the test set untouched until the final evaluation section.

0. Setup

This homework uses the modern R machine-learning ecosystem:

- `tidymodels` for splitting, preprocessing, modeling, prediction, and metrics.
- `themis` for imbalance-aware recipe steps such as upsampling.
- `readr` for loading the raw text files.
- `dplyr`, loaded through `tidymodels`, for data handling.

```
# Install packages if needed:
# install.packages(c("tidymodels", "themis", "readr"))
#install.packages( "themis")

library(tidymodels)
library(themis)
```

```
library(readr)

theme_set(theme_minimal())

set.seed(42)
```

1. Load the Dataset

The dataset has 36 numeric predictors. These represent four spectral bands measured across a 3 by 3 pixel neighborhood.

The original land-cover classes are:

Class	Land-Cover Label
1	red soil
2	cotton crop
3	grey soil
4	damp grey soil
5	soil with vegetation stubble
7	very damp grey soil

For this homework, class 4, damp grey soil, is the positive class.

```
feature_names <- paste0("band_", 1:36)
column_names <- c(feature_names, "land_cover")

train_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.trn"
test_url <- "https://archive.ics.uci.edu/ml/machine-learning-databases/statlog/satimage/sat.tst"

landsat_raw <- bind_rows(
  read_table(train_url, col_names = column_names, show_col_types = FALSE),
  read_table(test_url, col_names = column_names, show_col_types = FALSE)
)

landsat <- landsat_raw %>%
  mutate(
    damp_grey_soil = if_else(land_cover == 4, "damp_grey_soil", "other"),
    damp_grey_soil = factor(damp_grey_soil, levels = c("damp_grey_soil", "other"))
  ) %>%
  select(-land_cover)


landsat
```

```
## # A tibble: 6,435 x 37
##    band_1 band_2 band_3 band_4 band_5 band_6 band_7 band_8 band_9 band_10
##     <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>   <dbl>
##  1     92    115    120     94     84    102    106     79     84     102
##  2     84    102    106     79     84    102    102     83     80     102
##  3     84    102    102     83     80    102    102     79     84      94
##  4     80    102    102     79     84     94    102     79     80      94
##  5     84     94    102     79     80     94     98     76     80     102
##  6     80     94     98     76     80    102    102     79     76     102
##  7     76    102    106     83     76    102    106     87     80      98
```

```
##  8     76    102    106     87     80     98    106     79     76      94
##  9     76     89     98     76     76     94     98     76     76      98
## 10     76     94     98     76     76     98    102     72     76      94
## # i 6,425 more rows
## # i 27 more variables: band_11 <dbl>, band_12 <dbl>, band_13 <dbl>,
## #   band_14 <dbl>, band_15 <dbl>, band_16 <dbl>, band_17 <dbl>, band_18 <dbl>,
## #   band_19 <dbl>, band_20 <dbl>, band_21 <dbl>, band_22 <dbl>, band_23 <dbl>,
## #   band_24 <dbl>, band_25 <dbl>, band_26 <dbl>, band_27 <dbl>, band_28 <dbl>,
## #   band_29 <dbl>, band_30 <dbl>, band_31 <dbl>, band_32 <dbl>, band_33 <dbl>,
## #   band_34 <dbl>, band_35 <dbl>, band_36 <dbl>, damp_grey_soil <fct>
```

Question 1: Data Meaning and Prediction Target

Answer in 1-3 sentences.

- What is the model trying to predict?
- Why is this a classification problem rather than a regression problem?

Answer:
The model is trying to predict the ammount of damp grey soil in land cover from spectral bands. This is a classification problem becasue the combination of spectral bands that results in damp grey soil being identified is not linear.

2. Initial Data Checks

Before modeling, inspect the dataset. Verify the number of rows and columns, check data types, look for missing values, and summarize the numeric predictors.

```
# TODO: inspect the dataset shape, column names, data types, missing values, and numeric summaries.
glimpse(landsat)
```

```
## Rows: 6,435
## Columns: 37
## $ band_1         <dbl> 92, 84, 84, 80, 84, 80, 76, 76, 76, 76, 76, 72, 72, 76,~
## $ band_2         <dbl> 115, 102, 102, 102, 94, 94, 102, 102, 89, 94, 98, 94, 8~
## $ band_3         <dbl> 120, 106, 102, 102, 102, 98, 106, 106, 98, 98, 102, 90,~
## $ band_4         <dbl> 94, 79, 83, 79, 79, 76, 83, 87, 76, 76, 72, 72, 76, 76,~
## $ band_5         <dbl> 84, 84, 80, 84, 80, 80, 76, 80, 76, 76, 76, 72, 72, 72,~
## $ band_6         <dbl> 102, 102, 102, 94, 94, 102, 102, 98, 94, 98, 94, 89, 89~
## $ band_7         <dbl> 106, 102, 102, 102, 98, 102, 106, 106, 98, 102, 90, 94,~
## $ band_8         <dbl> 79, 83, 79, 79, 76, 79, 87, 79, 76, 72, 76, 76, 76, 72,~
## $ band_9         <dbl> 84, 80, 84, 80, 80, 76, 80, 76, 76, 76, 76, 72, 76, 68,~
## $ band_10        <dbl> 102, 102, 94, 94, 102, 102, 98, 94, 98, 94, 89, 89, 94,~
## $ band_11        <dbl> 102, 102, 102, 98, 102, 102, 106, 102, 102, 90, 94, 98,~
## $ band_12        <dbl> 83, 79, 79, 76, 79, 79, 79, 76, 72, 76, 76, 76, 76, 72,~
## $ band_13        <dbl> 101, 92, 84, 84, 84, 76, 80, 80, 80, 76, 76, 76, 76, 71~
## $ band_14        <dbl> 126, 112, 103, 99, 99, 99, 107, 112, 95, 91, 95, 87, 87~
## $ band_15        <dbl> 133, 118, 104, 104, 104, 104, 118, 118, 104, 104, 100, ~
## $ band_16        <dbl> 103, 85, 81, 78, 81, 81, 88, 88, 74, 74, 78, 74, 67, 67~
## $ band_17        <dbl> 92, 84, 84, 84, 76, 76, 80, 80, 76, 76, 76, 76, 71, 68,~
## $ band_18        <dbl> 112, 103, 99, 99, 99, 99, 112, 107, 91, 95, 91, 87, 87,~
## $ band_19        <dbl> 118, 104, 104, 104, 104, 108, 118, 113, 104, 100, 100, ~
## $ band_20        <dbl> 85, 81, 78, 81, 81, 85, 88, 85, 74, 78, 74, 67, 70, 67,~
## $ band_21        <dbl> 84, 84, 84, 76, 76, 76, 80, 80, 76, 76, 76, 71, 71, 68,~
## $ band_22        <dbl> 103, 99, 99, 99, 99, 103, 107, 95, 95, 91, 87, 87, 83, ~
## $ band_23        <dbl> 104, 104, 104, 104, 108, 118, 113, 100, 100, 100, 100, ~
```

```
## $ band_24        <dbl> 81, 78, 81, 81, 85, 88, 85, 78, 78, 74, 74, 70, 67, 67,~
## $ band_25        <dbl> 102, 88, 84, 84, 84, 84, 79, 79, 75, 75, 79, 75, 75, 71~
## $ band_26        <dbl> 126, 121, 107, 99, 99, 103, 107, 103, 91, 91, 87, 91, 8~
## $ band_27        <dbl> 134, 128, 113, 104, 104, 104, 113, 104, 96, 96, 93, 96,~
## $ band_28        <dbl> 104, 100, 87, 79, 79, 79, 87, 83, 75, 71, 71, 71, 67, 6~
## $ band_29        <dbl> 88, 84, 84, 84, 84, 79, 79, 79, 75, 79, 79, 75, 71, 71,~
## $ band_30        <dbl> 121, 107, 99, 99, 103, 107, 103, 103, 91, 87, 87, 87, 8~
## $ band_31        <dbl> 128, 113, 104, 104, 104, 109, 104, 104, 96, 93, 93, 93,~
## $ band_32        <dbl> 100, 87, 79, 79, 79, 87, 83, 79, 71, 71, 67, 67, 67, 62~
## $ band_33        <dbl> 84, 84, 84, 84, 79, 79, 79, 79, 79, 79, 75, 71, 71, 67,~
## $ band_34        <dbl> 107, 99, 99, 103, 107, 107, 103, 95, 87, 87, 87, 87, 79~
## $ band_35        <dbl> 113, 104, 104, 104, 109, 109, 104, 100, 93, 93, 96, 89,~
## $ band_36        <dbl> 87, 79, 79, 79, 87, 87, 79, 79, 71, 67, 71, 67, 62, 62,~
## $ damp_grey_soil <fct> other, other, other, other, other, other, other, other,~
```

```
dim(landsat)
```

```
## [1] 6435   37
```

```
summary(landsat)
```

```
##      band_1          band_2          band_3          band_4
##  Min.   : 39.0   Min.   : 27.00   Min.   : 53.00   Min.   : 33.00
##  1st Qu.: 60.0   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
##  Median : 68.0   Median : 87.00   Median :101.00   Median : 81.00
##  Mean   : 69.4   Mean   : 83.59   Mean   : 99.29   Mean   : 82.59
##  3rd Qu.: 80.0   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.0   Max.   :137.00   Max.   :140.00   Max.   :154.00
##      band_5          band_6          band_7          band_8
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.0
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.0
##  Median : 68.00   Median : 85.00   Median :101.00   Median : 81.0
##  Mean   : 69.15   Mean   : 83.24   Mean   : 99.11   Mean   : 82.5
##  3rd Qu.: 80.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.0
##  Max.   :104.00   Max.   :137.00   Max.   :145.00   Max.   :157.0
##      band_9          band_10         band_11         band_12
##  Min.   : 40.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 68.00
##  Median : 67.00   Median : 85.00   Median :100.00   Median : 81.00
##  Mean   : 68.91   Mean   : 82.89   Mean   : 98.85   Mean   : 82.39
##  3rd Qu.: 79.00   3rd Qu.:102.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :130.00   Max.   :145.00   Max.   :157.00
##     band_13         band_14         band_15         band_16
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
##  Median : 68.00   Median : 85.00   Median :101.00   Median : 81.00
##  Mean   : 69.29   Mean   : 83.48   Mean   : 99.31   Mean   : 82.64
##  3rd Qu.: 80.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :137.00   Max.   :145.00   Max.   :154.00
##     band_17         band_18         band_19         band_20
##  Min.   : 40.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.0
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.0
##  Median : 68.00   Median : 85.00   Median :100.00   Median : 81.0
##  Mean   : 69.05   Mean   : 83.17   Mean   : 99.15   Mean   : 82.6
##  3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.0
```

```
##  Max.   :104.00   Max.   :130.00   Max.   :145.00   Max.   :157.0
##     band_21          band_22          band_23          band_24
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 68.00
##  Median : 67.00   Median : 84.00   Median :100.00   Median : 81.00
##  Mean   : 68.84   Mean   : 82.86   Mean   : 98.95   Mean   : 82.47
##  3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :130.00   Max.   :145.00   Max.   :157.00
##     band_25          band_26          band_27          band_28
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
##  Median : 68.00   Median : 85.00   Median :100.00   Median : 81.00
##  Mean   : 69.16   Mean   : 83.37   Mean   : 99.21   Mean   : 82.66
##  3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :131.00   Max.   :140.00   Max.   :154.00
##     band_29          band_30          band_31          band_32
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 69.00
##  Median : 68.00   Median : 85.00   Median :100.00   Median : 81.00
##  Mean   : 68.94   Mean   : 83.15   Mean   : 99.11   Mean   : 82.62
##  3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :130.00   Max.   :145.00   Max.   :157.00
##     band_33          band_34          band_35          band_36
##  Min.   : 39.00   Min.   : 27.00   Min.   : 50.00   Min.   : 29.00
##  1st Qu.: 60.00   1st Qu.: 71.00   1st Qu.: 85.00   1st Qu.: 68.00
##  Median : 67.00   Median : 84.00   Median :100.00   Median : 81.00
##  Mean   : 68.73   Mean   : 82.86   Mean   : 98.93   Mean   : 82.51
##  3rd Qu.: 79.00   3rd Qu.:103.00   3rd Qu.:113.00   3rd Qu.: 92.00
##  Max.   :104.00   Max.   :130.00   Max.   :145.00   Max.   :157.00
##         damp_grey_soil
##  damp_grey_soil: 626
##  other         :5809
##
##
##
##
```

```
colSums(is.na(landsat))
```

```
##         band_1         band_2         band_3         band_4         band_5
##              0              0              0              0              0
##         band_6         band_7         band_8         band_9        band_10
##              0              0              0              0              0
##        band_11        band_12        band_13        band_14        band_15
##              0              0              0              0              0
##        band_16        band_17        band_18        band_19        band_20
##              0              0              0              0              0
##        band_21        band_22        band_23        band_24        band_25
##              0              0              0              0              0
##        band_26        band_27        band_28        band_29        band_30
##              0              0              0              0              0
##        band_31        band_32        band_33        band_34        band_35
##              0              0              0              0              0
##        band_36 damp_grey_soil
##              0              0
```

```r
unique(landsat_raw$land_cover)
```

```
## [1] 3 4 5 7 2 1
```

```r
table(landsat$damp_grey_soil)
```

```
##
## damp_grey_soil          other
##            626           5809
```

```r
# Hints: dim, glimpse, colSums, is.na, summary
```

Question 2: Predictor Variables

- Are the predictors mostly numeric or categorical?

Answer:
The predictors are all numeric values.

3. Quantify Class Imbalance

Calculate the number and percentage of observations in each class. Then calculate the majority-to-minority ratio.

```r
# calculate class counts and percentages.

count(landsat, damp_grey_soil)
```

```
## # A tibble: 2 x 2
##   damp_grey_soil     n
##   <fct>          <int>
## 1 damp_grey_soil   626
## 2 other           5809
```

```r
count(landsat, damp_grey_soil) %>% mutate(percent = n / sum(n) * 100)
```

```
## # A tibble: 2 x 3
##   damp_grey_soil     n percent
##   <fct>          <int>   <dbl>
## 1 damp_grey_soil   626    9.73
## 2 other           5809   90.3
```

```r
# calculate the majority-to-minority class ratio.
class_counts <- count(landsat, damp_grey_soil)
max(class_counts$n) / min(class_counts$n)
```

```
## [1] 9.279553
```

```r
# sensitivity / recall for a model that says all majority class
# TP/(TP+FN)
# true positives would be the damp grey soil? Since that is what we are after?
# true negatives would be all the "others", false negatives are "626" damp grey soil
0/(0+626)
```

```
## [1] 0
```

Question 3: Why Accuracy Can Mislead

Answer in 2-3 sentences.

- If a model predicted the majority class for every row, why might its accuracy look acceptable?
- What would that model's recall be for damp grey soil?
- Why is recall important when mapping a rare class?

Answer:
If the model predicted majority class for every row, the majority class is "other", so it would have a 90.3% accuracy which would seem very good. The recall would be 0, because there are no true positives of the value we are trying to model and 626 false negatives.

4. Create Train, Validation, and Test Sets

Use three disjoint subsets:

- **Training set:** fit preprocessing parameters and model parameters.
- **Validation set:** compare models and select a probability threshold.
- **Test set:** estimate final performance once, after all modeling choices are locked.

Use a 60% / 20% / 20% split. Because the positive class is uncommon, use stratification.

```
set.seed(42)
# TODO: create train_data, validation_data, and test_data.
# Use initial_split() twice and stratify by damp_grey_soil.

 dev_test_split <- initial_split(landsat, strata=damp_grey_soil, prop=0.8)
 # reserve 20 percent of data for testing
 dev_data <- training(dev_test_split)
test_data <- testing(dev_test_split)
 dim(landsat)
```

```
## [1] 6435   37
```

```
 dim(test_data)
```

```
## [1] 1287   37
```

```
 6435*.2
```

```
## [1] 1287
```

```
 6435-1287
```

```
## [1] 5148
```

```
dim(dev_test_split)
```

```
##   analysis assessment          n          p
##       5148       1287       6435         37
```

```
dim(dev_data)
```

```
## [1] 5148   37
```

```
#save the other 20% of data for validating, which after with holding 20% fro testing, 25% needs to be w
1287/5148 #20 percent of og data
```

```
## [1] 0.25
```

```
train_val_split <- initial_split(dev_data, strata=damp_grey_soil, prop=0.75)
train_data <- training(train_val_split)
validation_data <- testing(train_val_split)
```

```r
# checking the proportions
dim(train_data)[1]/dim(landsat)[1]
```

```
## [1] 0.6
```

```r
dim(test_data)[1]/dim(landsat)[1]
```

```
## [1] 0.2
```

```r
dim(validation_data)[1]/dim(landsat)[1]
```

```
## [1] 0.2
```

```r
# Hints:
# dev_test_split <- initial_split(...)
# dev_data <- training(...)
# test_data <- testing(...)
# train_val_split <- initial_split(...)
```

```r
# A split summary showing rows, positive count, and positive fraction
# for train, validation, and test.

split_summary <- bind_rows(
  train_data %>%
    summarize(
      split = "train",
      rows = n(),
      positive_count = sum(damp_grey_soil == "damp_grey_soil"),
      positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
    ),
  validation_data %>%
    summarize(
      split = "validation",
      rows = n(),
      positive_count = sum(damp_grey_soil == "damp_grey_soil"),
      positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
    ),
  test_data %>%
    summarize(
      split = "test",
      rows = n(),
      positive_count = sum(damp_grey_soil == "damp_grey_soil"),
      positive_fraction = mean(damp_grey_soil == "damp_grey_soil")
    )
)

split_summary
```

```
## # A tibble: 3 x 4
##   split       rows positive_count positive_fraction
##   <chr>      <int>          <int>             <dbl>
## 1 train       3861            360            0.0932
## 2 validation  1287            146            0.113
## 3 test        1287            120            0.0932
```

Question 4: Splitting and Leakage

Answer each prompt in 2-4 sentences.

1. Why is stratification useful in this dataset?
2. What is the validation set used for?
3. Why should the test set not be used to choose a model or threshold?
4. Explain how fitting scaling, imputation, or upsampling before splitting can leak information.

Answers:

1. Stratification is useful in this dataset to make the testing, validation, and training datasets have even proportions of damp grey soil, the desired variable. This makes the process of validating and testing more accurate, because with such few damp grey soils, if the data sets had no observations or all the observations of damp grey soils, it would be difficult to evaluate the models performance.
2. The validation set is used to evaluate the model as you are building it, without using the test data. Then make tweaks as needed without using the testing data.
3. The test data should not be used until the final model evaluation because as the modeler you would be informing your model with information it does not currently have. Then that would be a form of data leakage, and the model may not perform as well as expected when new data is introduced.
4. Data leakage can happen from scaling, imputation, and upsampling when it happens before splitting because these methods use the whole available dataset to fill in missing values. Scaling adjusts the distribution of the variable, so scaling a column with all values rather than 80% or 60% will have an impact on the changes. Imputation similarly uses all avaliable data in the column to calculate the mean or median. Upscaling uses data that is close spatially to inform the missing value. All these methods with test data included would influence the assigned values and would likely test well, but would build a model with information that it should not have to be properly evaluated.

5. Build Logistic-Regression Workflows

We will compare three workflows:

1. A majority-class baseline model.
2. Ordinary logistic regression.
3. Logistic regression with upsampling applied inside the recipe.

Upsampling increases the number of minority-class examples in the training data by resampling them. It must occur inside the modeling workflow so that it is applied only to training data.

```
# Create a basic recipe for logistic regression.

logistic_recipe <- recipe(damp_grey_soil ~ ., data = train_data) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())

upsample_recipe <- logistic_recipe %>%
  step_upsample(damp_grey_soil, over_ratio = 1)
```

```
# Create a null model specification and a logistic regression specification.

null_spec <- null_model() %>%
   set_engine("parsnip") %>%
   set_mode("classification")

logistic_spec <- logistic_reg() %>%
   set_engine("glm") %>%
   set_mode("classification")
```

```r
# TODO: create three workflows named:
# baseline_workflow
baseline_workflow <- workflow() %>%
  add_model(null_spec) %>% # no recipe needed because the predictors do not need preprocessing
  add_formula(damp_grey_soil ~ .) #soil predicted by (~) all other columns in the data (.)

# logistic_workflow
logistic_workflow <- workflow() %>%
  add_recipe(logistic_recipe) %>%
  add_model(logistic_spec)


# upsampled_logistic_workflow
upsampled_logistic_workflow <- workflow() %>%
  add_recipe(upsample_recipe) %>%
  add_model(logistic_spec)


# Hint: use workflow(), add_recipe(), and add_model().
```

```r
# TODO: fit all three workflows on train_data.
baseline_fit <- fit(baseline_workflow, data=train_data)
#baseline_fit


logistic_fit <- fit(logistic_workflow , data=train_data)
# logistic_fit
# Degrees of Freedom: 3860 Total (i.e. Null);  3824 Residual
# Null Deviance:        2514
# Residual Deviance: 2088   AIC: 2162

upsampled_logistic_fit <- fit(upsampled_logistic_workflow, data=train_data)
#upsampled_logistic_fit
# Degrees of Freedom: 6947 Total (i.e. Null);  6911 Residual
# Null Deviance:        9632
# Residual Deviance: 7184   AIC: 7258
```

Question 5: Model Choice

Answer in 1-3 sentences.

- Why include a majority-class baseline?
- Why is logistic regression a reasonable first classification model?
- What does upsampling change?
- Why must upsampling happen inside the training workflow?

Answer:
A majority-class baseline seems like the most basic option for predicting soil type, and a valuable starting point for being able to compare models to predict better than missing all rare observations. We are working with a binary yes/no of damp grey soil, which makes a logistic regression reasonable. Upsampling provides the training data with more observations the minority class so the ratio of majority to minority is smaller, and can better predict occurences of the minority class. Upsampling has to happen in the training data, because otherwise it would be informed by the rare occurences we need to evaluate it against later in the test and validation data.

6. Evaluate Models on the Validation Set

For an imbalanced classification problem, no single metric is sufficient.

Important metrics:

- **Recall / sensitivity:** fraction of damp-grey-soil pixels detected.
- **Precision:** fraction of predicted damp-grey-soil pixels that were actually damp grey soil.
- **Specificity:** fraction of other land-cover pixels correctly identified.
- **Balanced accuracy:** mean of sensitivity and specificity.
- **F1:** balances precision and recall equally.
- **F2:** gives recall more weight than precision.
- **ROC-AUC:** ranking performance across thresholds.
- **PR-AUC:** precision-recall area; often useful for imbalanced data.

```
positive_level <- "damp_grey_soil"
prob_col <- ".pred_damp_grey_soil"

predict_with_threshold <- function(fit, new_data, threshold = 0.5) {
  # TODO: return a data frame with truth, predicted probabilities, and thresholded classes.
  df_predictions <- predict(fit, new_data, type="prob") %>%  # need type, otherwise just gives its pred
  bind_cols(new_data %>% select(damp_grey_soil))%>%
  mutate(.pred_class = if_else(.pred_damp_grey_soil >= threshold, "damp_grey_soil", "other"))%>%
  mutate(.pred_class = factor(.pred_class, levels = levels(damp_grey_soil)))
return(df_predictions)
  # Hints:
  # predict(...) #Only need to update this line!!
  # bind_cols(new_data %>% select(damp_grey_soil))
  # mutate(.pred_class = if_else(.pred_damp_grey_soil >= threshold, "damp_grey_soil", "other"))
  # mutate(.pred_class = factor(.pred_class, levels = levels(damp_grey_soil)))
}

calculate_metrics <- function(predictions) {
  tibble(
    accuracy = accuracy_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class
    ),
    recall = sens_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    precision = precision_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    specificity = spec_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    balanced_accuracy = bal_accuracy_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
```

```
      event_level = "first"
    ),
    f1 = f_meas_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      event_level = "first"
    ),
    f2 = f_meas_vec(
      predictions$damp_grey_soil,
      predictions$.pred_class,
      beta = 2,
      event_level = "first"
    ),
    roc_auc = roc_auc_vec(
      predictions$damp_grey_soil,
      predictions$.pred_damp_grey_soil,
      event_level = "first"
    ),
    pr_auc = pr_auc_vec(
      predictions$damp_grey_soil,
      predictions$.pred_damp_grey_soil,
      event_level = "first"
    )
  )
}
```

```
# TODO: calculate validation metrics for all three models at threshold 0.5.
baseline_val <- predict_with_threshold(baseline_fit, validation_data)
logistic_val <- predict_with_threshold(logistic_fit, validation_data)
upsampled_val <- predict_with_threshold(upsampled_logistic_fit, validation_data)

# Hints:
# baseline_val <- predict_with_threshold(...)
# logistic_val <- predict_with_threshold(...)
# upsampled_val <- predict_with_threshold(...)
#
bind_rows(
   calculate_metrics(baseline_val) %>% mutate(model = "Baseline"),
   calculate_metrics(logistic_val) %>% mutate(model = "Logistic regression"),
   calculate_metrics(upsampled_val) %>% mutate(model = "Upsampled logistic regression")
)
```

```
## Warning: While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
```

```
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
```

```
## # A tibble: 3 x 10
##   accuracy recall precision specificity balanced_accuracy      f1      f2
##      <dbl>  <dbl>     <dbl>       <dbl>             <dbl>   <dbl>   <dbl>
## 1    0.887 0         NA           1                 0.5   NA      NA
## 2    0.886 0.0274     0.444       0.996             0.512  0.0516  0.0337
## 3    0.642 0.801      0.213       0.621             0.711  0.337   0.516
## # i 3 more variables: roc_auc <dbl>, pr_auc <dbl>, model <chr>
```

```
# TODO: create confusion matrices for the two logistic-regression models.
head(baseline_val)
```

```
## # A tibble: 6 x 4
##   .pred_damp_grey_soil .pred_other damp_grey_soil .pred_class
##                  <dbl>       <dbl> <fct>          <fct>
## 1               0.0932       0.907 damp_grey_soil other
## 2               0.0932       0.907 other          other
## 3               0.0932       0.907 other          other
## 4               0.0932       0.907 other          other
## 5               0.0932       0.907 other          other
## 6               0.0932       0.907 other          other
```

```
#baseline for fun
conf_mat(data=baseline_val, truth=damp_grey_soil, estimate=.pred_class)
```

```
##                 Truth
## Prediction       damp_grey_soil other
##   damp_grey_soil              0     0
##   other                     146  1141
```

```
#logistic
conf_mat(data=logistic_val, truth=damp_grey_soil, estimate=.pred_class)
```

```
##                 Truth
## Prediction       damp_grey_soil other
##   damp_grey_soil              4     5
##   other                     142  1136
```

```
#upsampled logistic
conf_mat(data=upsampled_val, truth=damp_grey_soil, estimate=.pred_class)
```

```
##                 Truth
## Prediction       damp_grey_soil other
##   damp_grey_soil            117   432
##   other                      29   709
```

```
# Hint:
# conf_mat(...)
```

Question 6: Validation Comparison

Using the validation results, answer in complete sentences.

1. Why can the baseline model have high accuracy but zero usefulness for the positive class?
2. Compare ordinary logistic regression and upsampled logistic regression at threshold 0.5.
3. Which model has better recall for damp grey soil?

4. What happened to precision and specificity?

Answers:

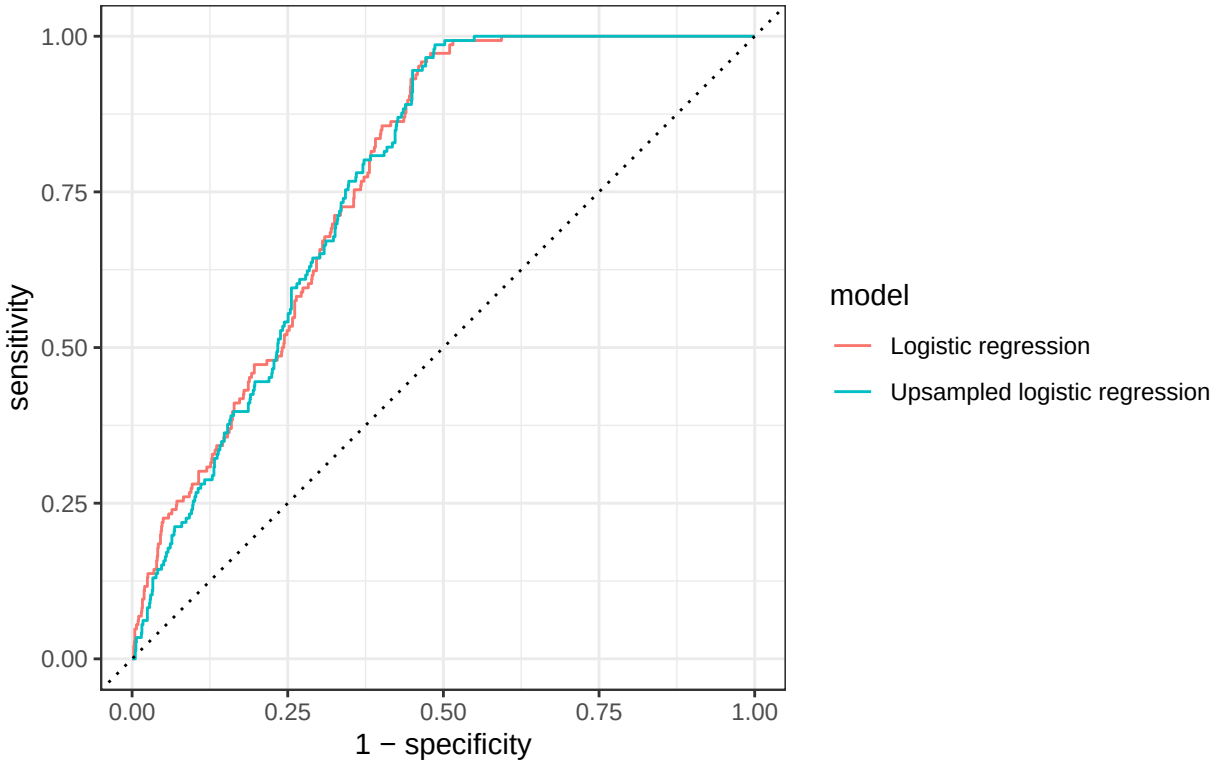
1. The baseline model has high accuracy because it correctly identifies the majority class, and misses all the rare occurances of the positive class of interest.
2. The logistic regression only correctly classifies one damp grey soil, and the upsampled classifies 108 correctly.
3. The upsampled has much better recall at 0.83 and the logisitic has a recall value of 0.008.
4. Precision and specificity are higher in the logistic regression, and lower in the upsampled because these penalize false positives and false negatives.

7. Examine ROC and Precision-Recall Curves

Curves show model behavior across possible thresholds. For imbalanced classification, the precision-recall curve is often more informative than the ROC curve because it focuses directly on positive-class detection.

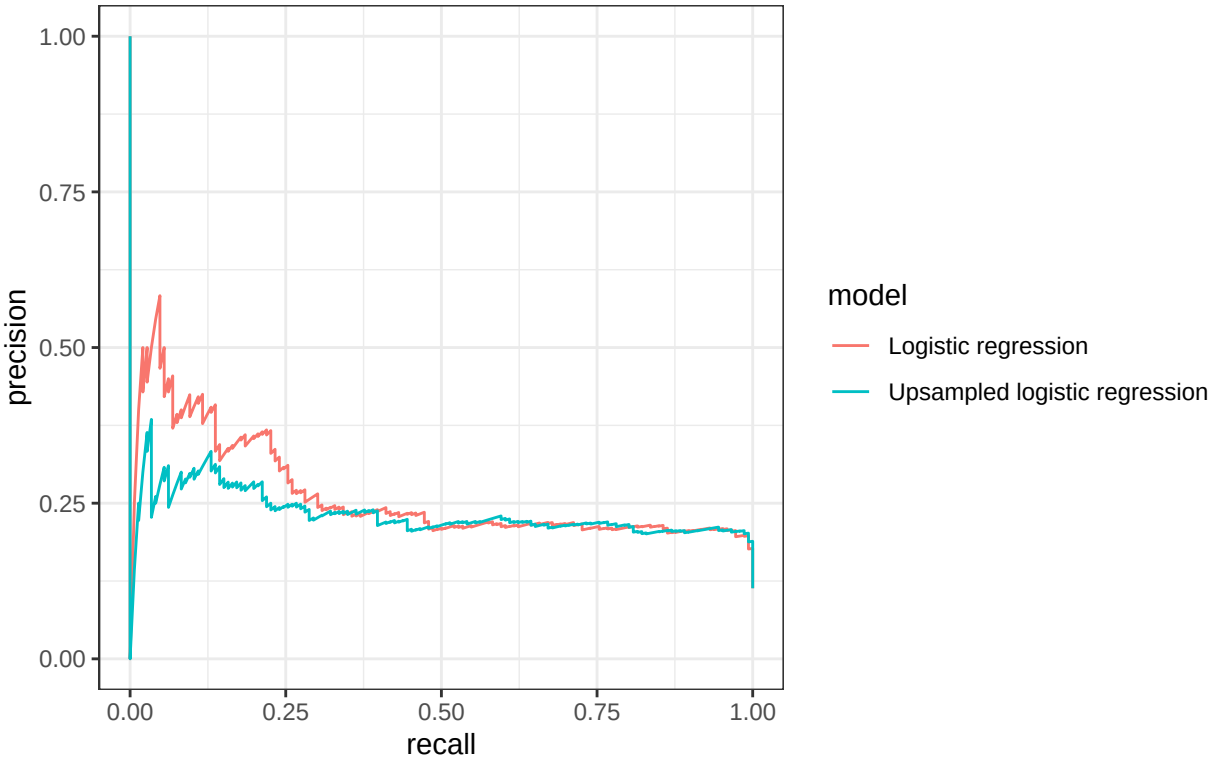
```
# plot ROC curves for ordinary and upsampled logistic regression.

bind_rows(
   logistic_val %>% mutate(model = "Logistic regression"),
   upsampled_val %>% mutate(model = "Upsampled logistic regression")
) %>%
  group_by(model) %>%
   roc_curve(truth = damp_grey_soil, .pred_damp_grey_soil, event_level = "first") %>%
   autoplot()
```




```
# TODO: plot precision-recall curves for ordinary and upsampled logistic regression.
bind_rows(
   logistic_val %>% mutate(model = "Logistic regression"),
   upsampled_val %>% mutate(model = "Upsampled logistic regression")
) %>%
```

```
  group_by(model) %>%
   pr_curve(truth = damp_grey_soil, .pred_damp_grey_soil, event_level = "first") %>%
   autoplot()
```



Question 7: Curve Interpretation

Answer in 3-6 sentences.

- Why is the precision-recall baseline equal to the positive-class prevalence?
- Can two models have similar ROC-AUC but different rare-class precision? Why?
- Which curve is more useful for thinking about missed rare land-cover areas versus false alarms?

Answer:
The precision-recall baseline is equal to the positive-class because it is measuring the number of positives which is very rare. The precision is high for accurately classifying and the recall is low. The similar ROC-AUC curves are due to all the true negatives being classified correctly as other, and the few correct rare event classifications do not make large changes in the curve proportionally. The precison recall graph is more useful here because it shows the trade-off of high accuracy versus the recall for rare events.

8. Test Probability Thresholds Using Validation Data

The default threshold of 0.5 is not automatically best for an imbalanced problem, but changing the threshold does not improve the model itself. It only changes the tradeoff between the types of errors the model makes.

For this exercise, test a range of thresholds for the **upsampled logistic regression** model and identify the threshold that maximizes **F2** on the validation set. Then compare that threshold with 0.5. If the F2-selected threshold creates too many false positives, lowers balanced accuracy, or makes the model less useful for the stated mapping goal, it is reasonable to conclude that threshold adjustment did not help.

The important lesson is not that threshold tuning always improves performance. The lesson is that thresholds encode a decision policy: they decide whether the model should miss more positive cases or produce more false alarms.

```r
# Search thresholds from 0.01 to 0.99 and identify the threshold with the best F2.

threshold_grid <- tibble(threshold = seq(0.01, 0.99, by = 0.01))
threshold_results <- threshold_grid %>%
   mutate(predictions = map(threshold, ~ predict_with_threshold(upsampled_logistic_fit, validation_data
   mutate(metrics = map(predictions, calculate_metrics)) %>%
   select(-predictions) %>%
   unnest(metrics)
```

```
## Warning: There were 6 warnings in `mutate()`.
## The first warning was:
## i In argument: `metrics = map(predictions, calculate_metrics)`.
## Caused by warning:
## ! While computing binary `precision()`, no predicted events were detected (i.e.
## `true_positive + false_positive = 0`).
## Precision is undefined in this case, and `NA` will be returned.
## Note that 146 true event(s) actually occurred for the problematic event level,
## damp_grey_soil
## i Run `dplyr::last_dplyr_warnings()` to see the 5 remaining warnings.
```

```r
best_threshold <- threshold_results %>%
   slice_max(f2, n = 1, with_ties = FALSE) %>%
   pull(threshold)

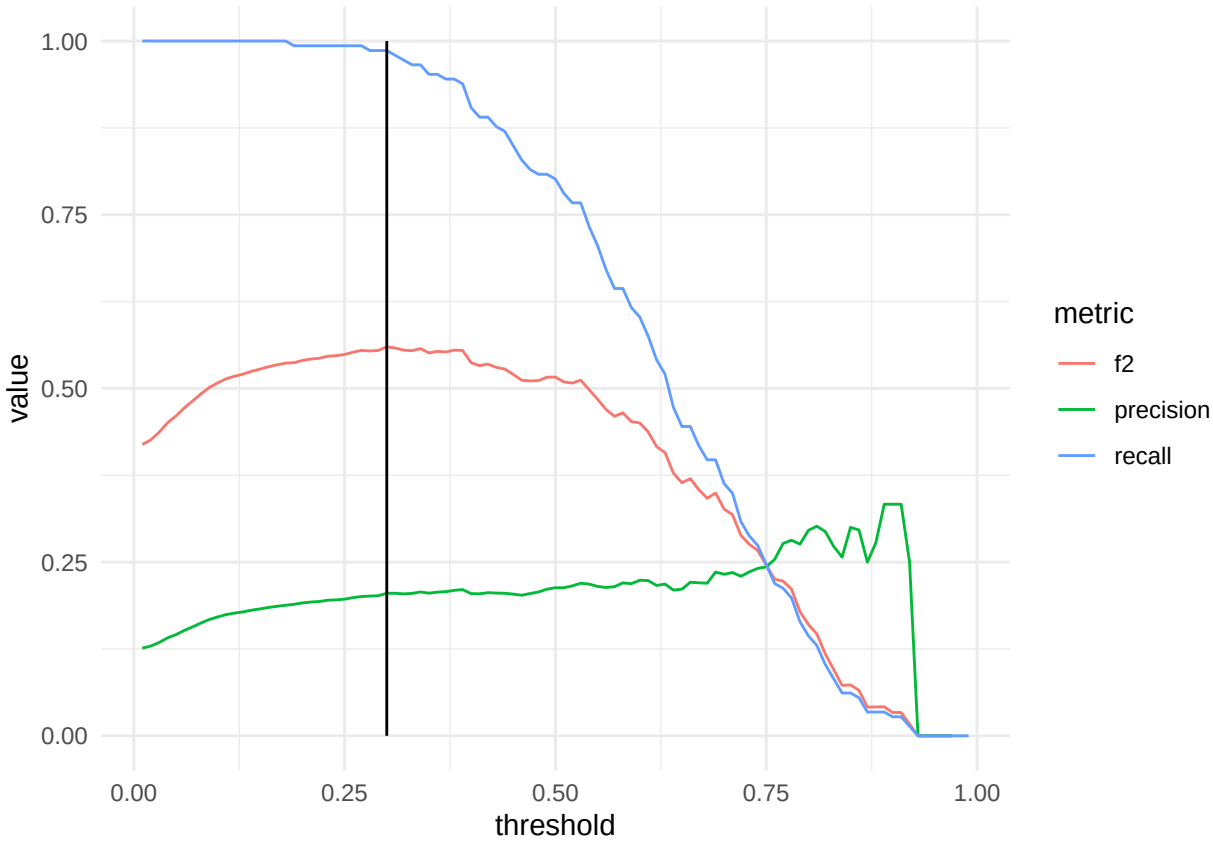
threshold_results[threshold_results$threshold ==0.34,]
```

```
## # A tibble: 1 x 10
##   threshold accuracy recall precision specificity balanced_accuracy    f1    f2
##       <dbl>    <dbl>  <dbl>     <dbl>       <dbl>             <dbl> <dbl> <dbl>
## 1      0.34    0.577  0.966     0.207       0.527             0.746 0.341 0.557
## # i 2 more variables: roc_auc <dbl>, pr_auc <dbl>
```

```r
# TODO: plot precision, recall, and F2 across thresholds.
threshold_results_long <- threshold_results %>%
  select(threshold, precision, recall, f2)%>%
  pivot_longer( cols=c(precision, recall, f2), names_to = "metric")

ggplot()+
  geom_line(data=threshold_results_long , aes(x=threshold, y=value, color=metric)) +
  geom_line(aes(x=best_threshold, y=c(0,1)))
```

```
## Warning: Removed 4 rows containing missing values or values outside the scale range
## (`geom_line()`).
```




```
# Hint: pivot precision, recall, and f2 into long format, then use ggplot().
```

Question 8: Threshold Choice

Answer in complete sentences.

1. Is your selected threshold above or below 0.5?
2. What happens to false positives and false negatives when the threshold is lowered?
3. Did the F2-selected threshold make the model more useful than the 0.5 threshold? Explain using at least two metrics or confusion-matrix counts.
4. Is maximizing F2 automatically the correct policy? What field, cost, or scientific information would you need before choosing a real operational threshold?

Answers:

1. The best threshold is 0.34.
2. False positives indicated by precision do not change very much when the threshold is lowered. False negatives are reduced largely when the threshold is lowered.
3. In comparison to the upsampled model, the 0.34 threshold, reduces accuracy from 0.61 to 0.55, improves recall from 0.84 to 0.99, precision is about the same, and specificity is lowered from 0.59 to 0.5. I would say yes, the model is more useful, recall is improved, with small reductions in accuracy and specificity.
4. I would need to know why we are interested in damp grey soil, and what is the real world cost to over estimating its presence. If this is exploratory for folks to go investigate and ground truth its precense, with limited funds overestimation may be detrimental to finding these areas of interest. If these are areas we need to protect or have plenty of money to ground truth, missing occurences and over estimating presence may be beneficial to the goals of the project.

9. Final Evaluation on the Untouched Test Set

At this point, the model and threshold have been selected using validation data. If the F2-selected threshold made validation performance worse for the assignment goal, you may keep the default 0.5 threshold instead. State which threshold you chose and why.

Do not use the test result to go back and choose a different model or threshold.

```
# Evaluate the upsampled logistic regression model on the test set
# using the threshold chosen from validation data.

# I am choosing to stick with the f2 threshold because it seems we care very much to find the damp grey

final_test_predictions <- predict_with_threshold(upsampled_logistic_fit, test_data, threshold = best_th
calculate_metrics(final_test_predictions)
```

```
## # A tibble: 1 x 9
##   accuracy recall precision specificity balanced_accuracy    f1    f2 roc_auc
##      <dbl>  <dbl>     <dbl>       <dbl>             <dbl> <dbl> <dbl>   <dbl>
## 1    0.531  0.983     0.164       0.484             0.734 0.281 0.492   0.770
## # i 1 more variable: pr_auc <dbl>
```

```
conf_mat(final_test_predictions, truth = damp_grey_soil, estimate = .pred_class)
```

```
##                 Truth
## Prediction       damp_grey_soil other
##   damp_grey_soil            118   602
##   other                       2   565
```

Question 9: Methods and Limitations

Answer in a short paragraph.

1. Compare upsampling with class weighting and random undersampling.
2. Where must resampling occur to avoid leakage when cross-validation is used?

Answer:
Upsampling resamples the training data to create more minority class occurences. Class weighting, uses all of the data in the ratio they were split, but durring model fitting it assigns a higher value to miss predicting the minority occurances. Random undersampling removes observations of the majority class randomly to balance the observations of majority and minority classes Resampling must occur in the training data workflow to avoid data leakage.

Grading Rubric (25 points)

Component	Points
Reproducibility, loading, and data checks	2
Class-imbalance analysis	3
Correct train/validation/test split and leakage explanation	3
Logistic-regression workflows	3
Baseline and model fitting	2
Validation metrics and interpretation	3
ROC and precision-recall curve interpretation	2
Threshold comparison using validation data	2
Final test evaluation	2
Scientific interpretation and limitations	3

Reproducibility Checklist

Before submitting, confirm that:

- ☐ The document knits without manual state from earlier sessions.
- ☐ All `TODO` sections are complete.
- ☐ Random seeds are fixed.
- ☐ Preprocessing is fitted only through workflows on training data.
- ☐ Upsampling occurs only inside the recipe/workflow.
- ☐ Validation data, not test data, are used for model and threshold decisions.
- ☐ The test set is evaluated only in the final section.
- ☐ Metrics are interpreted for the positive class, damp grey soil.
- ☐ Figures have readable titles and axis labels.
- ☐ Scientific limitations and error costs are discussed.

Dataset Citation

Srinivasan, A. (1993). Statlog (Landsat Satellite) [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55887